

xSigmaDex

smart contracts preliminary audit report (for internal use only)

Prepared for:

xSigma Lab

Authors: HashEx audit team

October 2021

Contents

Disclaimer	3
Introduction	4
Contracts overview	4
Found issues	5
Conclusion	11
References	11
Appendix A. Issues' severity classification	12
Appendix B. List of examined issue types	12
Appendix C. Delegation votes minting test	13
Appendix D. Slither static code analysis output	14

Disclaimer

This is a limited report on our findings based on our analysis, in accordance with good industry practice at the date of this report, in relation to cybersecurity vulnerabilities and issues in the framework and algorithms based on smart contracts, the details of which are set out in this report. In order to get a full view of our analysis, it is crucial for you to read the full report. While we have done our best in conducting our analysis and producing this report, it is important to note that you should not rely on this report and cannot claim against us on the basis of what it says or doesn't say, or how we produced it, and it is important for you to conduct your own independent investigations before making any decisions. We go into more detail on this in the disclaimer below – please make sure to read it in full.

DISCLAIMER: By reading this report or any part of it, you agree to the terms of this disclaimer. If you do not agree to the terms, please immediately cease reading this report, and delete and destroy any and all copies of this report downloaded and/or printed by you. This report is provided for information purposes only and on a non-reliance basis and does not constitute investment advice. No one shall have any right to rely on the report or its contents, and HashEx and its affiliates (including holding companies, shareholders, subsidiaries, employees, directors, officers, and other representatives) (HashEx) owe no duty of care towards you or any other person, nor does HashEx make any warranty or representation to any person on the accuracy or completeness of the report. The report is provided "as is", without any conditions, warranties, or other terms of any kind except as set out in this disclaimer, and HashEx hereby excludes all representations, warranties, conditions, and other terms (including, without limitation, the warranties implied by law of satisfactory quality, fitness for purpose and the use of reasonable care and skill) which, but for this clause, might have effect in relation to the report. Except and only to the extent that it is prohibited by law, HashEx hereby excludes all liability and responsibility, and neither you nor any other person shall have any claim against HashEx, for any amount or kind of loss or damage that may result to you or any other person (including without limitation, any direct, indirect, special, punitive, consequential or pure economic loss or damages, or any loss of income, profits, goodwill, data, contracts, use of money, or business interruption, and whether in delict, tort (including without limitation negligence), contract, breach of statutory duty, misrepresentation (whether innocent or negligent) or otherwise under any claim of any nature whatsoever in any jurisdiction) in any way arising from or connected with this report and the use, inability to use or the results of the use of this report, and any reliance on this report.

The analysis of the security is purely based on the smart contracts alone. No applications or operations were reviewed for security. No product code has been reviewed.

HashEx owns all copyright rights to the text, images, photographs, and other content provided in the following document. When using or sharing partly or in full, third parties must provide a direct link to the original document mentioning the author (<https://hashex.org>).

Introduction

HashEx was commissioned by the xSigma Lab team to perform an audit of their smart contracts. The audit was conducted between August 03 and August 12, 2021.

The code located in the GitHub repository @xsigma-dex-contracts/contracts/BSC was audited after the [439f277](#) commit. It must be noted that only 5 contracts in the BSC directory were the subject of this audit. The repository contains tests for audited contracts.

The purpose of this audit was to achieve the following:

- Identify potential security issues with smart contracts.
- Formally check the logic behind given smart contracts.

Information in this report should be used to understand the risk exposure of smart contracts, and as a guide to improving the security posture of smart contracts, by remediating the issues that were identified.

Update: xSigma team has responded to this report. Individual responses were added after each item in the [section](#). The updated code is located in the same repository after the [0563c2e](#) commit.

Contracts overview

`SigTokenBsc.sol`

Burnable ERC20 token with governance with limited minting under the owner's control. Further referred to as Token.

`SigMasterChefBsc.sol`

Farming contract with rewards in SigTokenBsc. Further referred to as MasterChef.

`CashBackBSC.sol`

Stores and transfers BNB cashback based on the caller's SigTokenBsc balance. Further referred to as Cashback.

`SigThreePoolProxyBsc.sol`

Proxy for the Curve/Ellipsis 3pool contract. Further referred to as Proxy.

`PeriodicDutchAuctionBsc.sol`

Dutch auction contract that buys user's SigTokenBsc token for stable coins. Further referred to as Auction.

Found issues

ID	Title	Severity	Response
01	Token: delegators not moved with transfers	High	Resolved
02	Token: mint is open for the owner	High	P/Resolved
03	MasterChef: not filtering input parameters in updateReferenceBlock	High	Resolved
04	Cashback: not filtering input parameters in setCashbackEther	High	Resolved
05	Proxy: not enough tests for solidity/vyper proxy	High	Resolved
06	MasterChef: _withUpdate is optional	Medium	Resolved
07	MasterChef: massUpdatePools gas limit	Medium	Acknowledged
08	MasterChef: allowed duplicated pools	Medium	Resolved
09	MasterChef: tokens with transfer fees are not supported	Medium	Resolved
10	Cashback: payCashbackEther allows contract trading	Medium	Acknowledged
11	Proxy: receive and fallback	Medium	Resolved
12	Auction: sellSigForStablecoin allows contract trading	Medium	Acknowledged
13	Token: checks uint >= 0	Low	Resolved
14	MasterChef: declaration with 0 assignment	Low	Acknowledged
15	MasterChef: lack of events	Low	Resolved
16	MasterChef: code for testing	Low	Acknowledged
17	Cashback: view function with tx parameters	Low	Acknowledged
18	Cashback: lack of error message	Low	Resolved
19	Proxy: variables naming	Low	Acknowledged

20	Proxy: lack of events	Low	Resolved
21	Auction: require in view function	Low	Resolved
22	Auction: unused parameter	Low	Resolved
23	Auction: result for ERC20 transfer not checked	Low	Resolved
24	Multiple: inconsistent comments & typos	Low	Resolved
25	General recommendations	Low	Resolved

#01 Token: delegators not moved with transfers High

`_moveDelegates()` function in [L234](#) of `SigTokenBsc` contract is designed to be used with every token transfer. However, in `SigTokenBsc` it's called with minting new tokens, but not with the usual transfers. The origin (`SushiToken`) has a warning ([L8](#)) about this issue. The issue allows to mint any number of delegation votes. A test for minting delegation votes can be seen in [Appendix C](#).

Recommendation: stick with the original governance logic and move delegates with transfers.

Update: the issue was fixed.

#02 Token: mint is open for the owner High

`mint()` function in [L95](#) of `SigToken Bsc` contract can be called by the owner or any address that the owner sets as a minter.

Recommendation: deny owner in `onlyMinter` and restrict owner's ability to set minters.

Update: the issue was partially fixed. `possibleToSetMinters` flag was added implementing a mechanism to prevent the owner from setting new minter accounts. Users would be able to check the approved minters and one-way `possibleToSetMinters` boolean variable.

#03 MasterChef: not filtering input parameters in `updateReferenceBlock` High

`updateReferenceBlockA()` and `updateReferenceBlockB()` functions in [L105-112](#) of `SigMasterChefBsc` contract should filter the input data. Otherwise, the owner is able to set such parameters that instantly mint the reward token to the limit.

Recommendation: add reasonable requirements for the new values of blocks A and B.

Update: the issue was fixed.

#04 Cashback: not filtering input parameters in setCashbackEther

High

setCashbackEther() function in [L49](#) of SigCashbackBsc contract should filter the input data. availableCashbackEther() function converts int to uint with arbitrary parameters set by the owner.

Recommendation: add reasonable requirements for setCashbackEther() parameters of refactor function to get rid of int variables.

Update: the issue was fixed.

#05 Proxy: not enough tests for solidity/vyper proxy

High

SigThreePoolProxyBsc contract is a Solidity proxy to a Curve/Ellipsis 3pool Vyper contract. Delegatecall proxies demand extra requirements to memory management, and mixing different compilers makes it even worse. We believe that the current amount of tests in xsigma-dex-contracts/test/SigThreePoolProxy.js is not sufficient as not all the lines of the Ellipsis.vy contract are covered.

Recommendation: increase test coverage or remove proxy in favor of modified Ellipsis/Curve contract with Cashback and Auction calls.

Update: the issue was fixed.

#06 MasterChef: _withUpdate is optional

Medium

add() and set() functions of SigMasterChefBsc contract have optional _withUpdate flag which calls massUpdatePools() if set true and may cause unfair rewards in case of rarely updated pools [\[1\]](#).

Update: the issue was fixed.

#07 MasterChef: massUpdatePools gas limit

Medium

for() loop over the pools array in massUpdatePools() [L228](#) function of SigMasterChefBsc contract may lead to block gas limit exceedance. This will lead to failure of changeFactor() and changeBscMintFraction1000() functions.

Recommendation: the owner should monitor the average gas consumption of massUpdatePools() and restrict the length of poolInfo[] array keeping in mind that block gas limit could be reduced in future network updates.

#08 MasterChef: allowed duplicated pools Medium

add() function [L166](#) allows adding a new pool with lpToken of the existing pool. It would mess up the rewards and wrong pools can't be removed or edited except the allocation.

Update: the issue was fixed.

#09 MasterChef: tokens with transfer fees are not supported Medium

SigMasterChefBsc contract doesn't support pools of tokens with fees on transfers as transfers don't check the resulting balance.

Update: the issue was fixed.

#10 Cashback: payCashbackEther allows contract trading Medium

payCashbackEther() function in [L118](#) of SigCashbackBSC contract does not restrict the contract calls making the algorithmic cashback calls possible including loaned funds. It may cause a situation in which all the cashback is taken by the contracts and bots, especially if a typical exchange() call takes less than 100k gas.

xSigma team response: this is by design, we would like to pay cashback to 1inch users and similar.

#11 Proxy: receive and fallback Medium

receive() in [L127](#) does not forward execution to the implementation and locks the ETH/BNB in the contract.

Update: the issue was fixed.

#12 Auction: sellSigForStablecoin allows contract trading Medium

sellSigForStablecoin() function in [L106](#) of PeriodicDutchAuctionBsc contract does not restrict the contract calls making the algorithmic calls including flashloaned funds possible.

#13 Token: checks uint >= 0 Low

maxRewardMintAfterBlocks() function of SigTokenBsc contract performs multiple checks on uint >= 0 in [L52, 55, 71](#).

Update: the issue was fixed.

#14 MasterChef: declaration with 0 assignment Low

Declaring variables with simultaneous 0 assignment [L66](#) of SigMasterChefBsc spends more gas than default declaring with the same result.

#15 MasterChef: lack of events Low

changeBscMintFraction1000(), updateReferenceBlockA(), updateReferenceBlockB(), add() and set() functions of SigmasterChefBsc contract should emit corresponding events.

Update: the issue was fixed.

#16 MasterChef: code for testing Low

BLOCK_NUMER_OFFSET constant [L80](#) is used only for testing and should be thoroughly removed from deploying scripts if the contract won't be refactored.

#17 Cashback: view function with tx parameters Low

Public view function entitledCashbackEther() [L90](#) of SigCashbackBSC contract uses transaction parameters and may return wrong results in explorers.

#18 Cashback: lack of error message Low

payCashbackEther() function of SigCashbackBSC contract contains no error message in [L119](#).

Update: the issue was fixed.

#19 Proxy: variables naming Low

Multiple variables of SigThreePoolProxyBsc contract are named in lowercase_with_underscores in contradiction with mixedCase suggested by Solidity naming conventions [\[2\]](#).

#20 Proxy: lack of events Low

changeAuction(), changeCashback() functions of SigThreePoolProxyBsc contract should emit corresponding events.

Update: the issue was fixed.

#21 Auction: require in view function Low

require statement [L89](#) should be avoided in view functions.

Update: the issue was fixed.

#22 Auction: unused parameter

Low

getSig1e18Price() function [L88](#) receives the tokenId parameters that are not in use after the latest commits.

Update: the issue was fixed.

#23 Auction: result for ERC20 transfer not checked

Low

The result of the ERC20 transfer in the PeriodicDutchAuctionBsc ([L123](#)) is not checked. We recommend using the SafeERC20 library from OpenZeppelin.

Update: the issue was fixed.

#24 Multiple: inconsistent comments & typos

Low

SigMasterChefBsc: [L216-220](#) and all the Sushi comments.

SigCashbackBSC: [L45](#), [L79](#), [L103](#).

PeriodicDutchAuctionBsc: [L25](#), [L65](#).

Update: the issues were fixed.

#25 General recommendations

Low

We recommend fixing the pragma version in the CashBackBSC contract.

Function maxRewardMintAfterBlocks makes assert checks which are always true (uint >= 0).

The CashBackBSC, SigThreePoolProxyBsc contracts lack zero checks on input parameters in the constructor.

Some functions could be declared external instead of public (see [Appendix D](#)).

Update: the issues were fixed.

Conclusion

5 high severity issues were found. We recommend fixing them before deployment.

Audit includes recommendations on the code improving and preventing potential attacks.

Update: xSigma team has responded to this report. Most of the issues were fixed including all the high ones. Individual responses were added after each item in the [section](#). Updated contracts are located in the same repository after the [0563c2e](#) commit.

References

1. [Dracula Protocol Medium](#)
2. [Solidity Docs: Naming Conventions](#)

Appendix A. Issues' severity classification

We consider an issue critical, if it may cause unlimited losses or break the workflow of the contract and could be easily triggered.

High severity issues may lead to limited losses or break interaction with users or other contracts under very specific conditions.

Medium severity issues do not cause the full loss of functionality but break the contract logic.

Low severity issues are typically nonoptimal code, unused variables, errors in messages. Usually, these issues do not need immediate reactions.

Appendix B. List of examined issue types

Business logic overview

Functionality checks

Following best practices

Access control and authorization

Reentrancy attacks

Front-run attacks

DoS with (unexpected) revert

DoS with block gas limit

Transaction-ordering dependence

ERC/BEP and other standards violation

Unchecked math

Implicit visibility levels

Excessive gas usage

Timestamp dependence

Forcibly sending ether to a contract

Weak sources of randomness

Shadowing state variables

Usage of deprecated code

Appendix C. Delegation votes minting test

The code of the test in hardhat framework:

Alice gets 100 tokens and generates voting power for Carol of 300 tokens.

```
describe('Delegation power attack', async function () {
  it('should multiply delegation power on transfers', async function() {
    const [deployer, alice, bob, carol] = await ethers.getSigners()
    const SigTokenBsc = await ethers.getContractFactory("SigTokenBsc");
    const sigTokenBsc = await SigTokenBsc.deploy(100);

    await sigTokenBsc.delegate(carol.address);
    await sigTokenBsc.transfer(alice.address, 100);
    await sigTokenBsc.connect(alice).delegate(carol.address);
    await sigTokenBsc.connect(alice).transfer(bob.address, 100);
    await sigTokenBsc.connect(bob).delegate(carol.address);
    const votes = await sigTokenBsc.getCurrentVotes(carol.address);
    console.log('votes', votes.toString())
  })
});
```

The test output (Carol gets voting power of 300):

```
HairToken
  Delegation power attack
votes 300
  ✓ should multiply delegation power on transfers
```

Appendix D. Slither static code analysis output

contracts/CashBackBSC.sol:104:32: Warning: Unused function parameter. Remove or comment out the variable name to silence this warning.

```
function calcCashbackEther(int128 coinId, uint256 amount, uint256 txGasPrice) public view returns
(uint256) {
```

^-----^

contracts/CashBackBSC.sol:118:31: Warning: Unused function parameter. Remove or comment out the variable name to silence this warning.

```
function payCashbackEther(int128 coinId, uint256 amount) external {
```

^-----^

contracts/PeriodicDutchAuctionBsc.sol:88:49: Warning: Unused function parameter. Remove or comment out the variable name to silence this warning.

```
function getSig1e18Price(uint256 currBlock, uint256 tokenId) public view returns (uint256) {
```

^-----^

@openzeppelin/contracts/access/Ownable.sol:26:5: Warning: Visibility for constructor is ignored. If you want the contract to be non-deployable, making it "abstract" is sufficient.

```
constructor () internal {
```

^ (Relevant source part starts here and spans across multiple lines).

@openzeppelin/contracts/token/ERC20/ERC20.sol:55:5: Warning: Visibility for constructor is ignored. If you want the contract to be non-deployable, making it "abstract" is sufficient.

```
constructor (string memory name_, string memory symbol_) public {
```

^ (Relevant source part starts here and spans across multiple lines).

INFO:Detectors:

SigThreePoolProxyBsc.exchange2(int128,int128,uint256,uint256) (contracts/SigThreePoolProxyBsc.sol#153-167) uses delegatecall to a input-controlled function id

- (success,result) =

DELEGATION_TARGET.delegatecall(abi.encodeWithSignature(exchange(int128,int128,uint256,uint256),i,j,dx,min_dy)) (contracts/SigThreePoolProxyBsc.sol#154-157)

Reference: <https://github.com/crytic/slither/wiki/Detector-Documentation#controlled-delegatecall>

INFO:Detectors:

PeriodicDutchAuctionBsc.sellSigForStablecoin(uint256,uint256,uint256)

(contracts/PeriodicDutchAuctionBsc.sol#106-126) ignores return value by

IERC20(coins[buyTokenId]).transfer(msg.sender,outputAmount) (contracts/PeriodicDutchAuctionBsc.sol#123)

SigMasterChefBsc.safeSushiTransfer(address,uint256) (contracts/SigMasterChefBsc.sol#306-313) ignores

return value by SIG_TOKEN.transfer(_to,sushiBal) (contracts/SigMasterChefBsc.sol#309)

SigMasterChefBsc.safeSushiTransfer(address,uint256) (contracts/SigMasterChefBsc.sol#306-313) ignores

return value by SIG_TOKEN.transfer(_to,_amount) (contracts/SigMasterChefBsc.sol#311)

Reference: <https://github.com/crytic/slither/wiki/Detector-Documentation#unchecked-transfer>

INFO:Detectors:

CashBackBsc.entitledCashbackEther(uint256) (contracts/CashBackBSC.sol#90-101) performs a multiplication on the result of a division:

-gasUnitsCashback = (100_000 * cashBackLevel / CASHBACK_SIG_HIGH_AMOUNT)

(contracts/CashBackBSC.sol#99)

-gasUnitsCashback * gasPrice (contracts/CashBackBSC.sol#100)

SigMasterChefBsc.pendingSushi2(uint256,address,uint256) (contracts/SigMasterChefBsc.sol#195-223) performs a multiplication on the result of a division:

-totalSushiRewardForPool = totalSushiReward.mul(pool.allocPoint).div(totalAllocPoint)

(contracts/SigMasterChefBsc.sol#213)

-accSushiPerShare = accSushiPerShare.add(totalSushiRewardForPool.mul(1e12).div(lpSupply))

(contracts/SigMasterChefBsc.sol#214)

SigMasterChefBsc.updatePool(uint256) (contracts/SigMasterChefBsc.sol#234-254) performs a multiplication on the result of a division:

```
-totalSushiRewardForPool = totalSushiReward.mul(pool.allocPoint).div(totalAllocPoint)
(contracts/SigMasterChefBsc.sol#247)
-pool.accSushiPerShare =
```

```
pool.accSushiPerShare.add(totalSushiRewardForPool.mul(1e12).div(lpSupply))
(contracts/SigMasterChefBsc.sol#251)
```

Reference: <https://github.com/crytic/slither/wiki/Detector-Documentation#divide-before-multiply>
INFO:Detectors:

SigMasterChefBsc.updatePool(uint256) (contracts/SigMasterChefBsc.sol#234-254) uses a dangerous strict equality:

```
- lpSupply == 0 (contracts/SigMasterChefBsc.sol#240)
```

SigTokenBsc._writeCheckpoint(address,uint32,uint256,uint256) (contracts/SigTokenBsc.sol#254-265) uses a dangerous strict equality:

```
- nCheckpoints > 0 && checkpoints[delegatee][nCheckpoints - 1].fromBlock == blockNumber
```

```
(contracts/SigTokenBsc.sol#257)
```

Reference: <https://github.com/crytic/slither/wiki/Detector-Documentation#dangerous-strict-equalities>
INFO:Detectors:

Reentrancy in SigMasterChefBsc.add(uint256,IERC20,bool) (contracts/SigMasterChefBsc.sol#166-178):

External calls:

```
- massUpdatePools() (contracts/SigMasterChefBsc.sol#168)
- SIG_TOKEN.mint(address(this),totalSushiRewardForPool)
```

```
(contracts/SigMasterChefBsc.sol#249)
```

State variables written after the call(s):

```
- poolInfo.push(PoolInfo(_lpToken,_allocPoint,lastRewardBlock,0))
```

```
(contracts/SigMasterChefBsc.sol#172-177)
```

```
- totalAllocPoint = totalAllocPoint.add(_allocPoint) (contracts/SigMasterChefBsc.sol#171)
```

Reentrancy in SigMasterChefBsc.changeBscMintFraction1000(uint256) (contracts/SigMasterChefBsc.sol#93-97):

External calls:

```
- massUpdatePools() (contracts/SigMasterChefBsc.sol#95)
- SIG_TOKEN.mint(address(this),totalSushiRewardForPool)
```

```
(contracts/SigMasterChefBsc.sol#249)
```

State variables written after the call(s):

```
- bscMintFraction1000 = newBscMintFraction1000 (contracts/SigMasterChefBsc.sol#96)
```

Reentrancy in SigMasterChefBsc.changeFactor(uint256) (contracts/SigMasterChefBsc.sol#158-162):

External calls:

```
- massUpdatePools() (contracts/SigMasterChefBsc.sol#160)
- SIG_TOKEN.mint(address(this),totalSushiRewardForPool)
```

```
(contracts/SigMasterChefBsc.sol#249)
```

State variables written after the call(s):

```
- rewardTimeFactor = newFactor (contracts/SigMasterChefBsc.sol#161)
```

Reentrancy in SigMasterChefBsc.deposit(uint256,uint256) (contracts/SigMasterChefBsc.sol#257-274):

External calls:

```
- updatePool(_pid) (contracts/SigMasterChefBsc.sol#260)
- SIG_TOKEN.mint(address(this),totalSushiRewardForPool)
```

```
(contracts/SigMasterChefBsc.sol#249)
```

```
- safeSushiTransfer(msg.sender,pending) (contracts/SigMasterChefBsc.sol#264)
```

```
- SIG_TOKEN.transfer(_to,sushiBal) (contracts/SigMasterChefBsc.sol#309)
```

```
- SIG_TOKEN.transfer(_to,_amount) (contracts/SigMasterChefBsc.sol#311)
```

```
- pool.lpToken.safeTransferFrom(address(msg.sender),address(this),_amount)
```

```
(contracts/SigMasterChefBsc.sol#268)
```

State variables written after the call(s):

```
- user.amount = user.amount.add(_amount) (contracts/SigMasterChefBsc.sol#269)
```

```
- user.rewardDebt = user.amount.mul(pool.accSushiPerShare).div(1e12)
```

```
(contracts/SigMasterChefBsc.sol#272)
```

Reentrancy in SigMasterChefBsc.set(uint256,uint256,bool) (contracts/SigMasterChefBsc.sol#181-187):

External calls:

```
- massUpdatePools() (contracts/SigMasterChefBsc.sol#183)
```

```

- SIG_TOKEN.mint(address(this),totalSushiRewardForPool)
(contracts/SigMasterChefBsc.sol#249)
  State variables written after the call(s):
  - poolInfo[_pid].allocPoint = _allocPoint (contracts/SigMasterChefBsc.sol#186)
  - totalAllocPoint = totalAllocPoint.sub(poolInfo[_pid].allocPoint).add(_allocPoint)
(contracts/SigMasterChefBsc.sol#185)
Reentrancy in SigMasterChefBsc.updatePool(uint256) (contracts/SigMasterChefBsc.sol#234-254):
  External calls:
  - SIG_TOKEN.mint(address(this),totalSushiRewardForPool) (contracts/SigMasterChefBsc.sol#249)
  State variables written after the call(s):
  - pool.accSushiPerShare =
pool.accSushiPerShare.add(totalSushiRewardForPool.mul(1e12).div(lpSupply))
(contracts/SigMasterChefBsc.sol#251)
  - pool.lastRewardBlock = offsetBlockNumber() (contracts/SigMasterChefBsc.sol#253)
Reentrancy in SigMasterChefBsc.withdraw(uint256,uint256) (contracts/SigMasterChefBsc.sol#277-292):
  External calls:
  - updatePool(_pid) (contracts/SigMasterChefBsc.sol#281)
    - SIG_TOKEN.mint(address(this),totalSushiRewardForPool)
(contracts/SigMasterChefBsc.sol#249)
    - safeSushiTransfer(msg.sender,pending) (contracts/SigMasterChefBsc.sol#284)
    - SIG_TOKEN.transfer(_to,sushiBal) (contracts/SigMasterChefBsc.sol#309)
    - SIG_TOKEN.transfer(_to,_amount) (contracts/SigMasterChefBsc.sol#311)
  State variables written after the call(s):
  - user.amount = user.amount.sub(_amount) (contracts/SigMasterChefBsc.sol#287)
Reentrancy in SigMasterChefBsc.withdraw(uint256,uint256) (contracts/SigMasterChefBsc.sol#277-292):
  External calls:
  - updatePool(_pid) (contracts/SigMasterChefBsc.sol#281)
    - SIG_TOKEN.mint(address(this),totalSushiRewardForPool)
(contracts/SigMasterChefBsc.sol#249)
    - safeSushiTransfer(msg.sender,pending) (contracts/SigMasterChefBsc.sol#284)
    - SIG_TOKEN.transfer(_to,sushiBal) (contracts/SigMasterChefBsc.sol#309)
    - SIG_TOKEN.transfer(_to,_amount) (contracts/SigMasterChefBsc.sol#311)
  - pool.lpToken.safeTransfer(address(msg.sender),_amount) (contracts/SigMasterChefBsc.sol#288)
  State variables written after the call(s):
  - user.rewardDebt = user.amount.mul(pool.accSushiPerShare).div(1e12)
(contracts/SigMasterChefBsc.sol#290)

```

Reference: <https://github.com/crytic/slither/wiki/Detector-Documentation#reentrancy-vulnerabilities-1>
INFO:Detectors:

SigTokenBsc.maxRewardMintAfterBlocks(uint256) (contracts/SigTokenBsc.sol#41-73) contains a tautology or contradiction:

```
- assert(bool)(totalBonus >= 0) (contracts/SigTokenBsc.sol#52)
```

SigTokenBsc.maxRewardMintAfterBlocks(uint256) (contracts/SigTokenBsc.sol#41-73) contains a tautology or contradiction:

```
- assert(bool)(0 <= era) (contracts/SigTokenBsc.sol#55)
```

SigTokenBsc.maxRewardMintAfterBlocks(uint256) (contracts/SigTokenBsc.sol#41-73) contains a tautology or contradiction:

```
- assert(bool)(0 <= result) (contracts/SigTokenBsc.sol#71)
```

Reference: <https://github.com/crytic/slither/wiki/Detector-Documentation#tautology-or-contradiction>
INFO:Detectors:

CashBackBsc.constructor(SigTokenBsc,address)._caller (contracts/CashBackBSC.sol#18) lacks a zero-check on :

```
- caller = _caller (contracts/CashBackBSC.sol#21)
```

SigThreePoolProxyBsc.constructor(address,address[3],address,uint256,uint256,uint256,address,address,address,SigTokenBsc,CashBackBsc)._owner (contracts/SigThreePoolProxyBsc.sol#61) lacks a zero-check on :

```
- owner = _owner (contracts/SigThreePoolProxyBsc.sol#83)
```

SigThreePoolProxyBsc.constructor(address,address[3],address,uint256,uint256,uint256,address,address,address,SigTokenBsc,CashBackBsc)._pool_token (contracts/SigThreePoolProxyBsc.sol#63) lacks a zero-check on :

```
- token = _pool_token (contracts/SigThreePoolProxyBsc.sol#86)
```

SigThreePoolProxyBsc.constructor(address,address[3],address,uint256,uint256,uint256,address,address,address,SigTokenBsc,CashBackBsc)._fee_converter (contracts/SigThreePoolProxyBsc.sol#67) lacks a zero-check on :

- fee_converter = _fee_converter (contracts/SigThreePoolProxyBsc.sol#87)

SigThreePoolProxyBsc.constructor(address,address[3],address,uint256,uint256,uint256,address,address,address,SigTokenBsc,CashBackBsc)._delegationTarget (contracts/SigThreePoolProxyBsc.sol#69) lacks a zero-check on :

- DELEGATION_TARGET = _delegationTarget (contracts/SigThreePoolProxyBsc.sol#89)

SigThreePoolProxyBsc.constructor(address,address[3],address,uint256,uint256,uint256,address,address,address,SigTokenBsc,CashBackBsc)._dutchAuction (contracts/SigThreePoolProxyBsc.sol#70) lacks a zero-check on :

- dutchAuction = _dutchAuction (contracts/SigThreePoolProxyBsc.sol#90)

SigThreePoolProxyBsc.changeAuction(address).newAuction (contracts/SigThreePoolProxyBsc.sol#140) lacks a zero-check on :

- dutchAuction = newAuction (contracts/SigThreePoolProxyBsc.sol#142)

Reference: <https://github.com/crytic/slither/wiki/Detector-Documentation#missing-zero-address-validation>

INFO:Detectors:

SigThreePoolProxyBsc.withdraw_admin_fees_to_auction() (contracts/SigThreePoolProxyBsc.sol#130-138) has external calls inside a loop: value = coin.balanceOf(address(this)) - balances[i] (contracts/SigThreePoolProxyBsc.sol#133)

SigThreePoolProxyBsc.donate_admin_fees2() (contracts/SigThreePoolProxyBsc.sol#175-180) has external calls inside a loop: balances[i] = IERC20(coins[i]).balanceOf(address(this)) (contracts/SigThreePoolProxyBsc.sol#178)

Reference: <https://github.com/crytic/slither/wiki/Detector-Documentation/#calls-inside-a-loop>

INFO:Detectors:

Reentrancy in SigMasterChefBsc.deposit(uint256,uint256) (contracts/SigMasterChefBsc.sol#257-274):

External calls:

- updatePool(_pid) (contracts/SigMasterChefBsc.sol#260)
 - SIG_TOKEN.mint(address(this),totalSushiRewardForPool) (contracts/SigMasterChefBsc.sol#249)
- safeSushiTransfer(msg.sender,pending) (contracts/SigMasterChefBsc.sol#264)
 - SIG_TOKEN.transfer(_to,sushiBal) (contracts/SigMasterChefBsc.sol#309)
 - SIG_TOKEN.transfer(_to,_amount) (contracts/SigMasterChefBsc.sol#311)
- pool.lpToken.safeTransferFrom(address(msg.sender),address(this),_amount) (contracts/SigMasterChefBsc.sol#268)

Event emitted after the call(s):

- Deposit(msg.sender,_pid,_amount) (contracts/SigMasterChefBsc.sol#273)

Reentrancy in SigMasterChefBsc.emergencyWithdraw(uint256) (contracts/SigMasterChefBsc.sol#295-303):

External calls:

- pool.lpToken.safeTransfer(address(msg.sender),amount) (contracts/SigMasterChefBsc.sol#301)

Event emitted after the call(s):

- EmergencyWithdraw(msg.sender,_pid,amount) (contracts/SigMasterChefBsc.sol#302)

Reentrancy in PeriodicDutchAuctionBsc.sellSigForStablecoin(uint256,uint256,uint256) (contracts/PeriodicDutchAuctionBsc.sol#106-126):

External calls:

- sigToken.burnFrom(msg.sender,sellSigAmount) (contracts/PeriodicDutchAuctionBsc.sol#121)
- IERC20(coins[buyTokenId]).transfer(msg.sender,outputAmount) (contracts/PeriodicDutchAuctionBsc.sol#123)

Event emitted after the call(s):

- Bought(msg.sender,sellSigAmount,buyTokenId,buyAtLeastAmount,outputAmount) (contracts/PeriodicDutchAuctionBsc.sol#125)

Reentrancy in SigMasterChefBsc.withdraw(uint256,uint256) (contracts/SigMasterChefBsc.sol#277-292):

External calls:

- updatePool(_pid) (contracts/SigMasterChefBsc.sol#281)
 - SIG_TOKEN.mint(address(this),totalSushiRewardForPool) (contracts/SigMasterChefBsc.sol#249)
- safeSushiTransfer(msg.sender,pending) (contracts/SigMasterChefBsc.sol#284)
 - SIG_TOKEN.transfer(_to,sushiBal) (contracts/SigMasterChefBsc.sol#309)
 - SIG_TOKEN.transfer(_to,_amount) (contracts/SigMasterChefBsc.sol#311)
- pool.lpToken.safeTransfer(address(msg.sender),_amount) (contracts/SigMasterChefBsc.sol#288)

Event emitted after the call(s):

- Withdraw(msg.sender,_pid,_amount) (contracts/SigMasterChefBsc.sol#291)

Reference: <https://github.com/crytic/slither/wiki/Detector-Documentation#reentrancy-vulnerabilities-3>

INFO:Detectors:

SigTokenBsc.delegateBySig(address,uint256,uint256,uint8,bytes32,bytes32)
(contracts/SigTokenBsc.sol#162-171) uses timestamp for comparisons

Dangerous comparisons:

- require(bool,string)(now <= expiry,SIG::delegateBySig: signature expired)
(contracts/SigTokenBsc.sol#169)

Reference: <https://github.com/crytic/slither/wiki/Detector-Documentation#block-timestamp>

INFO:Detectors:

Address.isContract(address) (node_modules/@openzeppelin/contracts/utils/Address.sol#26-35) uses assembly

- INLINE ASM (node_modules/@openzeppelin/contracts/utils/Address.sol#33)

Address._verifyCallResult(bool,bytes,string)
(node_modules/@openzeppelin/contracts/utils/Address.sol#171-188) uses assembly

- INLINE ASM (node_modules/@openzeppelin/contracts/utils/Address.sol#180-183)

SigThreePoolProxyBsc._delegate(address) (contracts/SigThreePoolProxyBsc.sol#101-121) uses assembly

- INLINE ASM (contracts/SigThreePoolProxyBsc.sol#103-120)

SigTokenBsc.getChainId() (contracts/SigTokenBsc.sol#272-276) uses assembly

- INLINE ASM (contracts/SigTokenBsc.sol#274)

Reference: <https://github.com/crytic/slither/wiki/Detector-Documentation#assembly-usage>

INFO:Detectors:

console._sendLogPayload(bytes) (node_modules/hardhat/console.sol#7-14) uses assembly

- INLINE ASM (node_modules/hardhat/console.sol#10-13)

Reference: <https://github.com/crytic/slither/wiki/Detector-Documentation#assembly-usage>

INFO:Detectors:

Different versions of Solidity is used:

- Version used: ['0.6.12', '>=0.6.0<0.8.0', '>=0.6.2<0.8.0', '^0.6.12']
- >=0.6.0<0.8.0 (node_modules/@openzeppelin/contracts/access/Ownable.sol#3)
- >=0.6.0<0.8.0 (node_modules/@openzeppelin/contracts/math/Math.sol#3)
- >=0.6.0<0.8.0 (node_modules/@openzeppelin/contracts/math/SafeMath.sol#3)
- >=0.6.0<0.8.0 (node_modules/@openzeppelin/contracts/token/ERC20/ERC20.sol#3)
- >=0.6.0<0.8.0 (node_modules/@openzeppelin/contracts/token/ERC20/ERC20Burnable.sol#3)
- >=0.6.0<0.8.0 (node_modules/@openzeppelin/contracts/token/ERC20/IERC20.sol#3)
- >=0.6.0<0.8.0 (node_modules/@openzeppelin/contracts/token/ERC20/SafeERC20.sol#3)
- >=0.6.2<0.8.0 (node_modules/@openzeppelin/contracts/utils/Address.sol#3)
- >=0.6.0<0.8.0 (node_modules/@openzeppelin/contracts/utils/Context.sol#3)
- >=0.6.0<0.8.0 (node_modules/@openzeppelin/contracts/utils/EnumerableSet.sol#3)
- ^0.6.12 (contracts/CashBackBSC.sol#3)
- 0.6.12 (contracts/PeriodicDutchAuctionBsc.sol#3)
- 0.6.12 (contracts/SigMasterChefBsc.sol#8)
- 0.6.12 (contracts/SigThreePoolProxyBsc.sol#4)
- 0.6.12 (contracts/SigTokenBsc.sol#2)

Reference:

<https://github.com/crytic/slither/wiki/Detector-Documentation#different-pragma-directives-are-used>

INFO:Detectors:

Different versions of Solidity is used:

- Version used: ['>=0.4.22<0.9.0', '>=0.6.0<0.8.0', '>=0.6.2<0.8.0', '^0.7.0']
- >=0.6.0<0.8.0 (node_modules/@openzeppelin/contracts/access/Ownable.sol#3)
- >=0.6.0<0.8.0 (node_modules/@openzeppelin/contracts/math/Math.sol#3)
- >=0.6.0<0.8.0 (node_modules/@openzeppelin/contracts/math/SafeMath.sol#3)
- >=0.6.0<0.8.0 (node_modules/@openzeppelin/contracts/token/ERC20/ERC20.sol#3)
- >=0.6.0<0.8.0 (node_modules/@openzeppelin/contracts/token/ERC20/ERC20Burnable.sol#3)
- >=0.6.0<0.8.0 (node_modules/@openzeppelin/contracts/token/ERC20/IERC20.sol#3)
- >=0.6.0<0.8.0 (node_modules/@openzeppelin/contracts/token/ERC20/SafeERC20.sol#3)
- >=0.6.2<0.8.0 (node_modules/@openzeppelin/contracts/utils/Address.sol#3)
- >=0.6.0<0.8.0 (node_modules/@openzeppelin/contracts/utils/Context.sol#3)
- >=0.6.0<0.8.0 (node_modules/@openzeppelin/contracts/utils/EnumerableSet.sol#3)

- ^0.7.0 (contracts/Greeter.sol#2)
- >=0.4.22<0.9.0 (node_modules/hardhat/console.sol#2)

Reference:

<https://github.com/crytic/slither/wiki/Detector-Documentation#different-pragma-directives-are-used>

INFO:Detectors:

Pragma version>=0.6.0<0.8.0 (node_modules/@openzeppelin/contracts/access/Ownable.sol#3) is too complex

Pragma version>=0.6.0<0.8.0 (node_modules/@openzeppelin/contracts/math/Math.sol#3) is too complex

Pragma version>=0.6.0<0.8.0 (node_modules/@openzeppelin/contracts/math/SafeMath.sol#3) is too complex

Pragma version>=0.6.0<0.8.0 (node_modules/@openzeppelin/contracts/token/ERC20/ERC20.sol#3) is too complex

Pragma version>=0.6.0<0.8.0 (node_modules/@openzeppelin/contracts/token/ERC20/ERC20Burnable.sol#3) is too complex

Pragma version>=0.6.0<0.8.0 (node_modules/@openzeppelin/contracts/token/ERC20/IERC20.sol#3) is too complex

Pragma version>=0.6.0<0.8.0 (node_modules/@openzeppelin/contracts/token/ERC20/SafeERC20.sol#3) is too complex

Pragma version>=0.6.2<0.8.0 (node_modules/@openzeppelin/contracts/utils/Address.sol#3) is too complex

Pragma version>=0.6.0<0.8.0 (node_modules/@openzeppelin/contracts/utils/Context.sol#3) is too complex

Pragma version>=0.6.0<0.8.0 (node_modules/@openzeppelin/contracts/utils/EnumerableSet.sol#3) is too complex

Reference: <https://github.com/crytic/slither/wiki/Detector-Documentation#incorrect-versions-of-solidity>

INFO:Detectors:

Pragma version^0.7.0 (contracts/Greeter.sol#2) allows old versions

Pragma version>=0.4.22<0.9.0 (node_modules/hardhat/console.sol#2) is too complex

Reference: <https://github.com/crytic/slither/wiki/Detector-Documentation#incorrect-versions-of-solidity>

INFO:Detectors:

Low level call in Address.sendValue(address,uint256)

(node_modules/@openzeppelin/contracts/utils/Address.sol#53-59):

- (success) = recipient.call{value: amount}()

(node_modules/@openzeppelin/contracts/utils/Address.sol#57)

Low level call in Address.functionCallWithValue(address,bytes,uint256,string)

(node_modules/@openzeppelin/contracts/utils/Address.sol#114-121):

- (success, returndata) = target.call{value: value}(data)

(node_modules/@openzeppelin/contracts/utils/Address.sol#119)

Low level call in Address.functionStaticCall(address,bytes,string)

(node_modules/@openzeppelin/contracts/utils/Address.sol#139-145):

- (success, returndata) = target.staticcall(data)

(node_modules/@openzeppelin/contracts/utils/Address.sol#143)

Low level call in Address.functionDelegateCall(address,bytes,string)

(node_modules/@openzeppelin/contracts/utils/Address.sol#163-169):

- (success, returndata) = target.delegatecall(data)

(node_modules/@openzeppelin/contracts/utils/Address.sol#167)

Low level call in SigThreePoolProxyBsc.exchange2(int128,int128,uint256,uint256)

(contracts/SigThreePoolProxyBsc.sol#153-167):

- (success, result) =

DELEGATION_TARGET.delegatecall(abi.encodeWithSignature(exchange(int128,int128,uint256,uint256),i,j,dx,min_dy)) (contracts/SigThreePoolProxyBsc.sol#154-157)

Reference: <https://github.com/crytic/slither/wiki/Detector-Documentation#low-level-calls>

INFO:Detectors:

Parameter CashBackBsc.setCashbackEther(uint256,uint256,int256,int256)._startBlock

(contracts/CashBackBSC.sol#50) is not in mixedCase

Parameter CashBackBsc.setCashbackEther(uint256,uint256,int256,int256)._endBlock

(contracts/CashBackBSC.sol#51) is not in mixedCase

Parameter PeriodicDutchAuctionBsc.setAuctionSettings(uint256,uint256,uint256,uint256)._startBlock

(contracts/PeriodicDutchAuctionBsc.sol#73) is not in mixedCase

Parameter PeriodicDutchAuctionBsc.setAuctionSettings(uint256,uint256,uint256,uint256)._auctionPeriodBlocks

(contracts/PeriodicDutchAuctionBsc.sol#74) is not in mixedCase

Parameter PeriodicDutchAuctionBsc.setAuctionSettings(uint256,uint256,uint256,uint256)._lowestSig1e18Price

(contracts/PeriodicDutchAuctionBsc.sol#75) is not in mixedCase

Parameter PeriodicDutchAuctionBsc.setAuctionSettings(uint256,uint256,uint256,uint256)._highestSig1e18Price (contracts/PeriodicDutchAuctionBsc.sol#76) is not in mixedCase

Parameter SigMasterChefBsc.updateReferenceBlockA(uint256,uint256)._bscBlockA (contracts/SigMasterChefBsc.sol#105) is not in mixedCase

Parameter SigMasterChefBsc.updateReferenceBlockA(uint256,uint256)._ethBlockA (contracts/SigMasterChefBsc.sol#105) is not in mixedCase

Parameter SigMasterChefBsc.updateReferenceBlockB(uint256,uint256)._bscBlockB (contracts/SigMasterChefBsc.sol#109) is not in mixedCase

Parameter SigMasterChefBsc.updateReferenceBlockB(uint256,uint256)._ethBlockB (contracts/SigMasterChefBsc.sol#109) is not in mixedCase

Parameter SigMasterChefBsc.add(uint256,IERC20,bool)._allocPoint (contracts/SigMasterChefBsc.sol#166) is not in mixedCase

Parameter SigMasterChefBsc.add(uint256,IERC20,bool)._lpToken (contracts/SigMasterChefBsc.sol#166) is not in mixedCase

Parameter SigMasterChefBsc.add(uint256,IERC20,bool)._withUpdate (contracts/SigMasterChefBsc.sol#166) is not in mixedCase

Parameter SigMasterChefBsc.set(uint256,uint256,bool)._pid (contracts/SigMasterChefBsc.sol#181) is not in mixedCase

Parameter SigMasterChefBsc.set(uint256,uint256,bool)._allocPoint (contracts/SigMasterChefBsc.sol#181) is not in mixedCase

Parameter SigMasterChefBsc.set(uint256,uint256,bool)._withUpdate (contracts/SigMasterChefBsc.sol#181) is not in mixedCase

Parameter SigMasterChefBsc.pendingSushi(uint256,address)._pid (contracts/SigMasterChefBsc.sol#190) is not in mixedCase

Parameter SigMasterChefBsc.pendingSushi(uint256,address)._user (contracts/SigMasterChefBsc.sol#190) is not in mixedCase

Parameter SigMasterChefBsc.pendingSushi2(uint256,address,uint256)._pid (contracts/SigMasterChefBsc.sol#195) is not in mixedCase

Parameter SigMasterChefBsc.pendingSushi2(uint256,address,uint256)._user (contracts/SigMasterChefBsc.sol#195) is not in mixedCase

Parameter SigMasterChefBsc.updatePool(uint256)._pid (contracts/SigMasterChefBsc.sol#234) is not in mixedCase

Parameter SigMasterChefBsc.deposit(uint256,uint256)._pid (contracts/SigMasterChefBsc.sol#257) is not in mixedCase

Parameter SigMasterChefBsc.deposit(uint256,uint256)._amount (contracts/SigMasterChefBsc.sol#257) is not in mixedCase

Parameter SigMasterChefBsc.withdraw(uint256,uint256)._pid (contracts/SigMasterChefBsc.sol#277) is not in mixedCase

Parameter SigMasterChefBsc.withdraw(uint256,uint256)._amount (contracts/SigMasterChefBsc.sol#277) is not in mixedCase

Parameter SigMasterChefBsc.emergencyWithdraw(uint256)._pid (contracts/SigMasterChefBsc.sol#295) is not in mixedCase

Parameter SigMasterChefBsc.safeSushiTransfer(address,uint256)._to (contracts/SigMasterChefBsc.sol#306) is not in mixedCase

Parameter SigMasterChefBsc.safeSushiTransfer(address,uint256)._amount (contracts/SigMasterChefBsc.sol#306) is not in mixedCase

Variable SigMasterChefBsc.SIG_TOKEN (contracts/SigMasterChefBsc.sol#55) is not in mixedCase

Variable SigMasterChefBsc.BLOCK_NUMBER_OFFSET (contracts/SigMasterChefBsc.sol#80) is not in mixedCase

Function SigThreePoolProxyBsc.withdraw_admin_fees_to_auction() (contracts/SigThreePoolProxyBsc.sol#130-138) is not in mixedCase

Parameter SigThreePoolProxyBsc.exchange2(int128,int128,uint256,uint256).min_dy (contracts/SigThreePoolProxyBsc.sol#153) is not in mixedCase

Function SigThreePoolProxyBsc.donate_admin_fees2() (contracts/SigThreePoolProxyBsc.sol#175-180) is not in mixedCase

Variable SigThreePoolProxyBsc.admin_fee (contracts/SigThreePoolProxyBsc.sol#26) is not in mixedCase

Variable SigThreePoolProxyBsc.fee_converter (contracts/SigThreePoolProxyBsc.sol#29) is not in mixedCase

Variable SigThreePoolProxyBsc.initial_A (contracts/SigThreePoolProxyBsc.sol#32) is not in mixedCase

Variable SigThreePoolProxyBsc.future_A (contracts/SigThreePoolProxyBsc.sol#33) is not in mixedCase

Variable SigThreePoolProxyBsc.initial_A_time (contracts/SigThreePoolProxyBsc.sol#34) is not in mixedCase

Variable SigThreePoolProxyBsc.future_A_time (contracts/SigThreePoolProxyBsc.sol#35) is not in mixedCase

Variable SigThreePoolProxyBsc.admin_actions_deadline (contracts/SigThreePoolProxyBsc.sol#37) is not in mixedCase

Variable SigThreePoolProxyBsc.transfer_ownership_deadline (contracts/SigThreePoolProxyBsc.sol#38) is not in mixedCase

Variable SigThreePoolProxyBsc.future_fee (contracts/SigThreePoolProxyBsc.sol#39) is not in mixedCase

Variable SigThreePoolProxyBsc.future_admin_fee (contracts/SigThreePoolProxyBsc.sol#40) is not in mixedCase

Variable SigThreePoolProxyBsc.future_owner (contracts/SigThreePoolProxyBsc.sol#41) is not in mixedCase

Variable SigThreePoolProxyBsc.slot_fill_0 (contracts/SigThreePoolProxyBsc.sol#43) is not in mixedCase

Variable SigThreePoolProxyBsc.slot_fill_1 (contracts/SigThreePoolProxyBsc.sol#44) is not in mixedCase

Variable SigThreePoolProxyBsc.is_killed (contracts/SigThreePoolProxyBsc.sol#46) is not in mixedCase

Variable SigThreePoolProxyBsc.kill_deadline (contracts/SigThreePoolProxyBsc.sol#47) is not in mixedCase

Variable SigThreePoolProxyBsc.slot_fill_2 (contracts/SigThreePoolProxyBsc.sol#49) is not in mixedCase

Variable SigThreePoolProxyBsc.slot_fill_3 (contracts/SigThreePoolProxyBsc.sol#50) is not in mixedCase

Variable SigThreePoolProxyBsc.DELEGATION_TARGET (contracts/SigThreePoolProxyBsc.sol#54) is not in mixedCase

Variable SigThreePoolProxyBsc.SIG_TOKEN (contracts/SigThreePoolProxyBsc.sol#55) is not in mixedCase

Parameter SigTokenBsc.setMinter(address,bool)._minterAddress (contracts/SigTokenBsc.sol#90) is not in mixedCase

Parameter SigTokenBsc.setMinter(address,bool)._isMinter (contracts/SigTokenBsc.sol#90) is not in mixedCase

Parameter SigTokenBsc.mint(address,uint256)._to (contracts/SigTokenBsc.sol#95) is not in mixedCase

Parameter SigTokenBsc.mint(address,uint256)._amount (contracts/SigTokenBsc.sol#95) is not in mixedCase

Variable SigTokenBsc._delegates (contracts/SigTokenBsc.sol#108) is not in mixedCase

Reference:
<https://github.com/crytic/slither/wiki/Detector-Documentation#conformance-to-solidity-naming-conventions>

INFO:Detectors:

Parameter Greeter.setGreeting(string)._greeting (contracts/Greeter.sol#19) is not in mixedCase

Contract console (node_modules/hardhat/console.sol#4-1532) is not in CapWords

Reference:
<https://github.com/crytic/slither/wiki/Detector-Documentation#conformance-to-solidity-naming-conventions>

INFO:Detectors:

Redundant expression "this (node_modules/@openzeppelin/contracts/utils/Context.sol#21)" inContext (node_modules/@openzeppelin/contracts/utils/Context.sol#15-24)

Reference: <https://github.com/crytic/slither/wiki/Detector-Documentation#redundant-statements>

INFO:Detectors:

Variable SigMasterChefBsc.MAX_REWARD_TIME_FACTOR (contracts/SigMasterChefBsc.sol#73) is too similar to SigMasterChefBsc.MIN_REWARD_TIME_FACTOR (contracts/SigMasterChefBsc.sol#72)

Variable SigThreePoolProxyBsc.DELEGATION_TARGET (contracts/SigThreePoolProxyBsc.sol#54) is too similar to SigThreePoolProxyBsc.constructor(address,address[3],address,uint256,uint256,uint256,address,address,address,SigTokenBsc,CashBackBsc)._delegationTarget (contracts/SigThreePoolProxyBsc.sol#69)

Variable SigThreePoolProxyBsc.slot_fill_0 (contracts/SigThreePoolProxyBsc.sol#43) is too similar to SigThreePoolProxyBsc.slot_fill_1 (contracts/SigThreePoolProxyBsc.sol#44)

Variable SigThreePoolProxyBsc.slot_fill_0 (contracts/SigThreePoolProxyBsc.sol#43) is too similar to SigThreePoolProxyBsc.slot_fill_2 (contracts/SigThreePoolProxyBsc.sol#49)

Variable SigThreePoolProxyBsc.slot_fill_1 (contracts/SigThreePoolProxyBsc.sol#44) is too similar to SigThreePoolProxyBsc.slot_fill_2 (contracts/SigThreePoolProxyBsc.sol#49)

Variable SigThreePoolProxyBsc.slot_fill_2 (contracts/SigThreePoolProxyBsc.sol#49) is too similar to SigThreePoolProxyBsc.slot_fill_3 (contracts/SigThreePoolProxyBsc.sol#50)

Variable SigThreePoolProxyBsc.slot_fill_0 (contracts/SigThreePoolProxyBsc.sol#43) is too similar to SigThreePoolProxyBsc.slot_fill_3 (contracts/SigThreePoolProxyBsc.sol#50)

Variable SigThreePoolProxyBsc.slot_fill_1 (contracts/SigThreePoolProxyBsc.sol#44) is too similar to SigThreePoolProxyBsc.slot_fill_3 (contracts/SigThreePoolProxyBsc.sol#50)

Reference: <https://github.com/crytic/slither/wiki/Detector-Documentation#variable-names-are-too-similar>

INFO:Detectors:

console.slitherConstructorConstantVariables() (node_modules/hardhat/console.sol#4-1532) uses literals with too many digits:

setCashbackEther(uint256,uint256,int256,int256) should be declared external:
 - CashBackBsc.setCashbackEther(uint256,uint256,int256,int256) (contracts/CashBackBSC.sol#49-76)
 calcCashbackEther(int128,uint256,uint256) should be declared external:
 - CashBackBsc.calcCashbackEther(int128,uint256,uint256) (contracts/CashBackBSC.sol#104-115)
 setAuctionSettings(uint256,uint256,uint256,uint256) should be declared external:
 - PeriodicDutchAuctionBsc.setAuctionSettings(uint256,uint256,uint256,uint256)
 (contracts/PeriodicDutchAuctionBsc.sol#72-83)
 changeFactor(uint256) should be declared external:
 - SigMasterChefBsc.changeFactor(uint256) (contracts/SigMasterChefBsc.sol#158-162)
 add(uint256,IERC20,bool) should be declared external:
 - SigMasterChefBsc.add(uint256,IERC20,bool) (contracts/SigMasterChefBsc.sol#166-178)
 set(uint256,uint256,bool) should be declared external:
 - SigMasterChefBsc.set(uint256,uint256,bool) (contracts/SigMasterChefBsc.sol#181-187)
 deposit(uint256,uint256) should be declared external:
 - SigMasterChefBsc.deposit(uint256,uint256) (contracts/SigMasterChefBsc.sol#257-274)
 withdraw(uint256,uint256) should be declared external:
 - SigMasterChefBsc.withdraw(uint256,uint256) (contracts/SigMasterChefBsc.sol#277-292)
 emergencyWithdraw(uint256) should be declared external:
 - SigMasterChefBsc.emergencyWithdraw(uint256) (contracts/SigMasterChefBsc.sol#295-303)
 withdraw_admin_fees_to_auction() should be declared external:
 - SigThreePoolProxyBsc.withdraw_admin_fees_to_auction()
 (contracts/SigThreePoolProxyBsc.sol#130-138)
 exchange2(int128,int128,uint256,uint256) should be declared external:
 - SigThreePoolProxyBsc.exchange2(int128,int128,uint256,uint256)
 (contracts/SigThreePoolProxyBsc.sol#153-167)
 maxRewardMintAfterBlocks(uint256) should be declared external:
 - SigTokenBsc.maxRewardMintAfterBlocks(uint256) (contracts/SigTokenBsc.sol#41-73)
 setMinter(address,bool) should be declared external:
 - SigTokenBsc.setMinter(address,bool) (contracts/SigTokenBsc.sol#90-92)
 mint(address,uint256) should be declared external:
 - SigTokenBsc.mint(address,uint256) (contracts/SigTokenBsc.sol#95-99)
 Reference:
<https://github.com/crytic/slither/wiki/Detector-Documentation#public-function-that-could-be-declared-external>
 INFO:Detectors:
 name() should be declared external:
 - ERC20.name() (node_modules/@openzeppelin/contracts/token/ERC20/ERC20.sol#64-66)
 totalSupply() should be declared external:
 - ERC20.totalSupply() (node_modules/@openzeppelin/contracts/token/ERC20/ERC20.sol#96-98)
 balanceOf(address) should be declared external:
 - ERC20.balanceOf(address) (node_modules/@openzeppelin/contracts/token/ERC20/ERC20.sol#103-105)
 Reference:
<https://github.com/crytic/slither/wiki/Detector-Documentation#public-function-that-could-be-declared-external>
 INFO:Slither:. analyzed (27 contracts with 75 detectors), 171 result(s) found
 INFO:Slither:Use <https://crytic.io/> to get access to additional detectors and Github integration